

Módulo 04 — Skills vs. Agentes

Prompt, skill, agente ou decisão estruturada? O formato Agent Skills, o loop de um agente, as proteções e um framework de decisão, com o curador transformado em skill e em agente.

1. Visão geral
2. Conceitos
3. Exercício
4. Projeto final — etapa M4
5. Para revisar

Visão geral

Objetivos de aprendizagem

Ao final, a pessoa consegue:

1. explicar o que é um prompt, uma skill, um agente e uma decisão estruturada, e o que muda entre eles (quem decide os passos, custo, previsibilidade, como avaliar);
2. empacotar conhecimento como skill no formato aberto Agent Skills e explicar a divulgação progressiva;
3. descrever o loop de um agente (escolher ação → executar ferramenta → devolver resultado → repetir) e as proteções que ele precisa;
4. aplicar um framework de decisão a casos novos e justificar a escolha;
5. comparar com dados um pipeline fixo e um agente fazendo o mesmo trabalho.

Mensagem central

Comece pelo mais simples que resolve. **Skill** empacota o *como*; **agente** decide *o que fazer a seguir*. Autonomia é cara e imprevisível: só vale quando o próximo passo realmente depende do que se descobre. E mesmo assim, **o agente propõe; o código decide**.

Conceitos

Até aqui, o curador era um **pipeline fixo**: para cada notícia, o código chamava o modelo do mesmo jeito. Este módulo faz duas transformações e pergunta o que muda em cada uma:

- 1. empacotar o conhecimento do curador como uma **skill** reutilizável;
- 2. transformar o curador num **agente**, que decide sozinho quais fontes ler e o que avaliar.

No fim, o **framework de decisão** junta tudo: quando basta um prompt, quando empacotar como skill, quando construir um agente e quando usar uma decisão estruturada.

1. As quatro abordagens

	Prompt	Skill	Agente	Decisão estruturada
O que é	um pedido bem escrito, numa chamada	instruções + referências + scripts empacotados numa pasta reutilizável	um loop: o modelo escolhe a próxima ação, usa ferramentas e decide quando parar	uma escolha tipada de um conjunto fechado, com confiança (Módulo 2)
Quem decide os passos	você	você (a skill descreve como fazer)	o modelo	você
Ferramentas	não	opcional (scripts)	sim, é o centro	não
Custo e latência	1 chamada	1 chamada + o contexto da skill	variável: N passos + o que as ferramentas chamam	1 chamada, saída pequena
Previsibilidade	alta	alta	baixa (a trajetória muda)	alta
Como avaliar	eval da resposta	eval da resposta	eval da resposta e da trajetória	eval + calibração
Bom para	tarefa pontual de texto	conhecimento procedural reutilizado em vários lugares	tarefas abertas, em que o próximo passo depende do que se descobre	classificar, rotear, pontuar em volume

Elas se combinam: um agente pode **usar** uma skill; uma skill pode **conter** uma decisão estruturada; um pipeline fixo pode chamar um agente só para o pedaço aberto.

2. Skills

O formato

Uma **skill** é uma pasta com um `SKILL.md`: um cabeçalho YAML com `name` e `description`, seguido das instruções em Markdown. Pastas opcionais guardam o resto: `references/` (documentação), `scripts/` (código executável) e `assets/` (modelos, dados). A Anthropic lançou o formato em outubro de 2025 ([post](#)) e o publicou como padrão aberto em dezembro de 2025 ([especificação](#)). Outras ferramentas de agentes adotaram o mesmo formato.

Regras principais da especificação:

- `name`: até 64 caracteres, só minúsculas, números e hífens, **igual ao nome da pasta**;
- `description`: até 1024 caracteres, dizendo **o que a skill faz e quando usar**;
- corpo do `SKILL.md`: recomenda-se menos de 500 linhas; o detalhe vai para arquivos referenciados.

Divulgação progressiva

Um agente não lê todas as skills inteiras. Ele carrega em camadas:

1. **sempre**: só `name` + `description` de cada skill (~100 tokens cada);
2. **ao ativar**: o corpo do `SKILL.md`, quando a tarefa combina com a descrição;
3. **sob demanda**: os arquivos de `references/`, `scripts/`, `assets/`, só se precisar.

Por isso a `description` é a parte mais importante: se ela não tiver as palavras que aparecem nos pedidos, a skill nunca é ativada. **Analogia**: a lombada de um manual na estante. Você só abre o manual se o título bater com o problema, e só lê o apêndice se precisar.

Quando empacotar como skill

- o mesmo conhecimento é usado **de novo**, por pessoas ou ferramentas diferentes;
- há **conhecimento específico extenso** (checklists, guias, cláusulas, exemplos) que não cabe num prompt de uma linha;
- você quer **versionar** e revisar esse conhecimento como código.

Skill **não** dá autonomia: ela diz **como** fazer uma tarefa. Quem decide **quando** usá-la é a pessoa ou o agente.

No projeto, `skills/curar-noticia/` tem as instruções de classificação no `SKILL.md` e as definições das categorias em `references/guia-de-rotulagem.md`. O nosso código carrega a pasta e usa como prompt; a mesma pasta pode ser instalada num agente de código compatível.

3. Agentes

O loop

Um **agente** é um LLM num loop com ferramentas:

```
objetivo → [modelo escolhe a próxima ação] → executa a ferramenta → resultado volta  
para o modelo → ...  
  
"entregar" · atingir o limite  
  
até: responder · chamar
```

A diferença para um pipeline não é usar ferramentas; é **quem decide os passos**. A Anthropic separa os dois casos ([Building effective agents, 2024](#)):

- **workflows**: LLMs e ferramentas orquestrados por **caminhos definidos em código**;
- **agentes**: o LLM **dirige o próprio processo** e o uso de ferramentas.

A recomendação deles é começar simples ("otimizar chamadas únicas com recuperação e exemplos geralmente basta") e só usar agentes quando a troca vale a pena: agentes trocam **latência e custo** por desempenho em tarefas abertas. O mesmo texto lista padrões de workflow que resolvem muita coisa sem autonomia: encadeamento de prompts, roteamento, paralelização, orquestrador-trabalhadores e avaliador-otimizador.

A ideia de intercalar raciocínio e ação num loop ficou conhecida com o **ReAct** ([Yao et al., 2022](#)). Hoje, provedores e bibliotecas implementam esse loop com *tool calling*: o modelo devolve uma chamada estruturada ({ ferramenta, entrada }), o código executa e devolve o resultado. No projeto, `gerarComFerramentas` (@mentoria/llm) usa o loop do AI SDK; no exercício, você escreve o loop à mão.

Ferramentas bem desenhadas

As ferramentas são a interface entre o agente e o mundo. Boas práticas (o "ACI", *agent-computer interface*, do mesmo texto da Anthropic):

- **descrição clara** do que faz e de cada parâmetro: é o que o modelo lê para decidir;
- **entrada validada** por schema;
- **erros como mensagem**, não como exceção: "fonte desconhecida: X" deixa o modelo se corrigir;
- **idempotência**: chamar duas vezes não deve fazer estrago (no projeto, avaliar a mesma notícia de novo devolve a avaliação guardada);
- **menor privilégio**: o agente só recebe as ferramentas de que precisa. Ferramentas que **mudam** algo (publicar, apagar, pagar) pedem aprovação humana (Módulo 7).

O que muda ao virar agente

	Pipeline com skill	Agente
Custo	fixo e previsível	variável: depende da trajetória
Cobertura	lê tudo	pode pular fontes (economiza, mas pode perder notícias)
Trajetoária	sempre a mesma	pode mudar entre execuções
Novos modos de falha	—	parar sem entregar, repetir a mesma chamada, inventar ids, ler a fonte errada, seguir instruções vindas das ferramentas (injeção)
Avaliação	resultado	resultado e trajetória (quais ferramentas, quantos passos, custo)

Proteções

- **limite de passos** e **orçamento** de custo;
- **deteção de repetição** (a mesma chamada várias vezes seguidas);
- **validação no código** do que o agente entrega. No projeto, "o agente propõe; o código decide": um id que o agente não avaliou, ou cuja avaliação não foi "publicar", é recusado;
- **registro** de toda a trajetória, para depurar e avaliar;
- **humano no meio** antes de ações irreversíveis.

4. O framework de decisão

Responda, para o problema, nesta ordem:

1. **Os passos são variáveis e precisam de ferramentas?** (O próximo passo depende do que se descobre: investigar, pesquisar, navegar.) → **agente**, com limites e validação.
2. **A saída é uma decisão de um conjunto fechado, com passos fixos?** (Classificar, rotear, pontuar, aprovar.) → **decisão estruturada** com confiança.
3. **É reutilizado ou depende de conhecimento específico extenso?** → **skill**.
4. **Senão** → **prompt**.

E, em qualquer caso: **alto volume** pesa a favor do que é mais barato e previsível.

É uma heurística, não uma lei. Ela serve para **começar pelo mais simples** e só subir de nível quando houver um motivo que você consegue dizer em voz alta. O exercício transforma essas perguntas em código.

Quatro casos (os da aula)

Caso	Resposta	Por quê
Reescrever uma vez um comunicado do RH em linguagem simples	prompt	texto para pessoas, pontual, sem dados externos
Rotear 30 mil tickets/dia para 8 filas, duvidosos para humano	decisão estruturada	escolha de conjunto fechado, passos fixos, alto volume, confiança decide o fallback
Revisar contratos com o checklist jurídico , usado por vários times em ferramentas diferentes	skill	passos fixos, mas conhecimento extenso que precisa ser o mesmo em todo lugar
Investigar um incidente olhando logs, métricas e deploys	agente	o próximo passo depende do que o anterior revelou; cada passo usa uma ferramenta

Referências

- Anthropic (2024). *Building effective agents*. <https://www.anthropic.com/engineering/building-effective-agents>
- Anthropic (2025). *Equipping agents for the real world with Agent Skills*. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- Agent Skills — especificação. <https://agentskills.io/specification> · repositório: <https://github.com/agentskills/agentskills>
- Yao et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. <https://arxiv.org/abs/2210.03629>
- AI SDK — agentes: <https://ai-sdk.dev/docs/agents/overview> · controle do loop: <https://ai-sdk.dev/docs/agents/loop-control>

Exercício

Tarefa assíncrona. Os testes rodam offline: não precisa de modelo rodando.

Duas peças:

1. o **framework de decisão** da aula (prompt, skill, agente ou decisão estruturada?) como uma função testável;
2. um **agente escrito à mão**: o loop que qualquer biblioteca de agentes faz por baixo. O "modelo" é uma função que você controla nos testes.

Como rodar

Da raiz do repositório:

```
pnpm -F @mentoria/ex04-agentes test          # versão base
pnpm -F @mentoria/ex04-agentes test:desafio   # desafio extra
```

Os testes **falham** até você implementar. A solução de referência está em [solucao/](#). Tente antes de abrir. `pnpm -F @mentoria/ex04-agentes test:solucao` roda os mesmos testes contra ela.

Versão base

Arquivo	Função	O que faz
<code>src/framework.ts</code>	<code>recomendarAbordagem</code>	aplica as 4 regras do framework, na ordem, e devolve a abordagem com os motivos
<code>src/loop.ts</code>	<code>executarAgente</code>	o loop: decidir → executar ferramenta → registrar → repetir, até responder ou chegar ao limite

`src/casos.ts` (fornecido) tem os 4 casos da aula com a resposta esperada. Os testes usam esses casos.

Dicas:

- No framework, a **ordem** das regras importa: uma decisão que precisa investigar com ferramentas é agente, não decisão estruturada.
- No loop, erro de ferramenta **não** derruba o agente: vira um evento no histórico para o "modelo" ver na próxima volta.

Feito = `pnpm -F @mentoria/ex04-agentes test` com todos os testes passando.

Desafio extra — proteções

Em `src/desafio/protecoes.ts`, implemente `executarAgenteProtegido`: o mesmo loop, que também para quando o modelo **repete** a mesma chamada várias vezes seguidas e quando a próxima ferramenta **estouraria o orçamento** de custo. São as proteções que faltam a um agente de verdade além do limite de passos.

Feito = `pnpm -F @mentoria/ex04-agentes test:desafio` com todos os testes passando.

Para ir além (sem teste): troque o "modelo" de roteiro por um LLM de verdade. Monte o `decidir` com `gerarObjeto` (de `@mentoria/llm`) pedindo um objeto `{ tipo, nome, entrada, texto }` a partir do histórico. O que dá errado primeiro?

Perguntas para pensar

1. Pense num caso do seu trabalho que daria vontade de resolver com um agente. Ele passa na regra 1 do framework (passos variáveis e ferramentas)? Se não passa, qual abordagem mais simples resolveria?
2. Por que devolver o erro da ferramenta para o modelo, em vez de lançar uma exceção?
3. No seu loop, o que impede o agente de chamar uma ferramenta que não deveria? E de chamar uma permitida com uma entrada perigosa?
4. O limite de passos protege contra o quê? E o que ele **não** protege?

Projeto final — etapa M4

Etapa M4 — o curador como skill e como agente

Objetivo: empacotar o conhecimento do curador como uma **skill** reutilizável e, depois, transformar o curador num **agente** que decide sozinho quais fontes ler. Medir o que muda.

As notícias agora chegam por **fontes** simuladas (`dados/fontes.json` : dez fontes, cada uma com descrição e notícias).

1. **A skill** (`skills/curar-noticia/`), no formato aberto [Agent Skills](#):
 - escreva o `SKILL.md`: `name` igual à pasta, `description` dizendo o que faz **e quando usar**, e as instruções da classificação no corpo;
 - ponha as definições das categorias em `references/guia-de-rotulagem.md` e aponte para ele no corpo.
2. `src/m04/skill.ts`: `lerSkill(pasta)` (lê e valida o frontmatter segundo a especificação), `montarPromptDaSkill(skill, noticia)` e `classificarComSkill(noticia)`.
3. `src/m04/ferramentas.ts`: `criarFerramentas(deps)` → `listarFontes`, `lerFonte`, `avaliarNoticias` e `entregarSelecao`, com um registro do que o agente fez.
4. `src/m04/agente.ts`: `validarSelecao` (o agente propõe; o código decide) e `curarComAgente`, que roda o loop com `gerarComFerramentas` de `@mentoria/llm`.

Feito =

1. `pnpm -F @mentoria/curador exec vitest run test/m04.test.ts verde` (offline, com modelo simulado);
2. a comparação roda com o modelo de verdade e grava `saidas/m04-comparacao.md`:

```
pnpm -F @mentoria/curador m04:comparar      # pipeline com a skill x agente (2 execuções)
```

Abra o relatório e responda (vamos discutir na Aula 5):

- O agente leu todas as fontes? Quais pulou, e isso custou notícias?
- Quantas chamadas ao modelo cada abordagem fez? E o tempo?
- As duas execuções do agente seguiram a mesma trajetória?
- O código recusou algo que o agente entregou? Por quê?

Desafio extra: instale a sua skill num agente de código compatível com Agent Skills (por exemplo, copiando a pasta para o diretório de skills da ferramenta) e peça para ele classificar uma notícia nova. A `description` foi suficiente para a skill ser ativada?

Para revisar

Os 4 casos da aula

Para a turma classificar na discussão. Respostas e justificativas em `exercicio/src/casos.ts` e em `conceitos.md`.

1. **Reescrever um comunicado:** o RH precisa reescrever, uma única vez, um comunicado sobre a mudança de horário, em linguagem mais simples.
2. **Rotear tickets de suporte:** uma loja on-line recebe ~30 mil tickets por dia para 8 filas, com os duvidosos indo para uma pessoa.
3. **Revisar contratos com o checklist jurídico:** checklist de 40 itens e cláusulas-modelo, usado por vários times no chat interno e no editor de código.
4. **Investigar um incidente de produção:** o tempo de resposta de uma API subiu; é preciso olhar logs, métricas e deploys até achar a causa provável.

Perguntas para discussão (15 min)

Primeiro os 4 casos (votação rápida, depois `demo:casos --revelar`). Depois:

1. O agente do curador leu menos fontes e mesmo assim gastou mais. Em que cenário ele ganharia do pipeline?
2. Que caso do seu trabalho parece pedir um agente? Ele passa na primeira pergunta do framework (passos variáveis e ferramentas)?
3. Uma skill e um prompt longo num arquivo: qual a diferença na prática?
4. "O agente propõe; o código decide." O que o código deveria checar antes de aceitar o resultado de um agente no seu caso?
5. Como você avaliaria um agente, além de olhar a resposta final?

Armadilhas comuns

- **Começar pelo agente** porque é o mais interessante. Comece pelo prompt e suba de nível só com motivo.
- **Chamar de agente um pipeline com ferramentas.** Se o caminho está no código, é um workflow (e isso é bom).
- **Description vaga na skill** ("ajuda com notícias"): a skill nunca é ativada. Diga o que faz e quando usar.
- **SKILL.md gigante:** mova o detalhe para `references/` e aponte para ele.
- **Ferramenta que lança exceção** em vez de devolver uma mensagem de erro: o agente não consegue se corrigir.

- **Confiar no que o agente entrega** sem validar no código.
- **Agente sem limite** de passos e de custo.
- **Avaliar só a resposta final.** Num agente, a trajetória (o que leu, o que pulou, quanto gastou) também é resultado.